Simple Exponential Smoothing

Theory, statistical model and the `nabia` implementation

nabia documentation

October 2026

Abstract

This note documents `nabia::forecast::SimpleExponentialSmoothing`. We define the recursion and prove that it is an exponentially weighted average whose weights sum to one. We compute its mean lag and derive the span convention $\alpha = 2/(N+1)$. We then give the statistical model behind it, the innovations state-space model ETS(A,N,N), and derive the forecast distribution, including the variance $\sigma^2(1+(h-1)\alpha^2)$. We also prove its equivalence with ARIMA(0,1,1) and with the steady-state Kalman filter of the local level model. Finally, we describe the implementation: initialization, variance estimate, prediction intervals and complexity.

Contents

1 The recursion **1**
1.1 Exponential weights 2

2 The statistical model: ETS(A,N,N) **2**
2.1 Equivalence with ARIMA(0,1,1) 3
2.2 Equivalence with the Kalman filter of the local level model 3

3 Estimation **4**

4 Implementation **4**
4.1 Interface 4
4.2 Algorithm 4

1 The recursion

Let $y_0, \dots, y_{T-1}$ be an equally spaced series and $\alpha \in (0, 1]$ the smoothing factor. Simple exponential smoothing (SES) [1] maintains a *level* ℓ_t, updated as

$$\ell_t = \alpha\, y_t + (1-\alpha)\, \ell_{t-1}, \qquad t = 0, \dots, T-1, \tag{1}$$

from an initial level ℓ_{-1}. Equivalently, in *error-correction form*,

$$e_t = y_t - \ell_{t-1}, \qquad \ell_t = \ell_{t-1} + \alpha\, e_t, \tag{2}$$

where e_t is the one-step-ahead forecast error: ℓ_{t-1} is the forecast of y_t made at time $t-1$. Every forecast made at the end of the sample equals the final level,

$$\hat{y}_{T-1+h|T-1} = \ell_{T-1}, \qquad h = 1, 2, \dots$$

SES is therefore suited to series without trend or seasonality: it estimates the current level and extrapolates it flat. The value of α sets the trade-off between reacting to recent data ($\alpha \to 1$, the naive forecast $\ell_t = y_t$) and smoothing out noise ($\alpha \to 0$, an almost constant level).

1.1 Exponential weights

Proposition 1.1 (Exponentially weighted average)**.** *For $t \geq 0$,*

$$\ell_t = \sum_{j=0}^{t} \alpha(1-\alpha)^j \, y_{t-j} + (1-\alpha)^{t+1} \, \ell_{-1}, \tag{3}$$

and the weights add up to one: $\sum_{j=0}^{t} \alpha(1-\alpha)^j + (1-\alpha)^{t+1} = 1$.

Proof. By induction on t. For $t = 0$, (1) gives $\ell_0 = \alpha y_0 + (1-\alpha)\ell_{-1}$, which is (3). If (3) holds for $t-1$, then

$$\ell_t = \alpha y_t + (1-\alpha)\Big[\sum_{j=0}^{t-1} \alpha(1-\alpha)^j y_{t-1-j} + (1-\alpha)^t \ell_{-1}\Big] = \alpha y_t + \sum_{j=1}^{t} \alpha(1-\alpha)^j y_{t-j} + (1-\alpha)^{t+1}\ell_{-1},$$

which is (3) for t. For the sum, the geometric series gives $\sum_{j=0}^{t} \alpha(1-\alpha)^j = \alpha \, \frac{1-(1-\alpha)^{t+1}}{1-(1-\alpha)} = 1 - (1-\alpha)^{t+1}$. □

So the level is a weighted average of past observations, with weights that decay geometrically with age. The weight on the initial level ℓ_{-1} vanishes as t grows, so the choice of initialization matters only for the first $O(1/\alpha)$ steps.

Proposition 1.2 (Mean lag and the span convention)**.** *In the limit $t \to \infty$, the weights $w_j = \alpha(1-\alpha)^j$ have mean age $\bar{\jmath} = \sum_j j \, w_j = (1-\alpha)/\alpha$. A simple moving average over the last N points has mean age $(N-1)/2$. The two match when $\alpha = 2/(N+1)$.*

Proof. Differentiating the geometric series $\sum_{j \geq 0} q^j = 1/(1-q)$ gives $\sum_{j \geq 0} j q^{j-1} = 1/(1-q)^2$. With $q = 1-\alpha$, $\sum_j j \, \alpha q^j = \alpha q/(1-q)^2 = (1-\alpha)/\alpha$. The moving average puts weight $1/N$ on ages $0, \ldots, N-1$, so its mean age is $(N-1)/2$. Setting $(1-\alpha)/\alpha = (N-1)/2$ and solving gives $\alpha = 2/(N+1)$. □

This is the convention behind `EWMA::alpha_from_span` in `nabia/signal`. The mean lag is also the delay with which the level follows a linear trend: if $y_t = a + bt$, the steady-state level is $\ell_t = a + b(t - \bar{\jmath})$. This lag is why trend strategies built on SES or EWMA signals react late, and why trend models (local linear trend in the Kalman filter note) extrapolate the slope instead.

2 The statistical model: ETS(A,N,N)

On its own, (1) is only an algorithm. To attach uncertainty to the forecasts we need a statistical model whose optimal one-step predictor is SES. The standard choice is the innovations state-space model with additive error, no trend and no seasonality, ETS(A,N,N) [4]:

$$y_t = \ell_{t-1} + \varepsilon_t, \qquad \ell_t = \ell_{t-1} + \alpha \, \varepsilon_t, \qquad \varepsilon_t \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2). \tag{4}$$

A single error source drives both equations (hence "innovations" form). Given ℓ_{t-1}, the conditional mean of y_t is ℓ_{t-1}, and $\varepsilon_t = y_t - \ell_{t-1} = e_t$. Substituting into the state equation gives exactly (2). So the SES levels are the model's states, and the SES errors are its innovations.

Theorem 2.1 (Forecast distribution)**.** *Under* (4)*, conditionally on the information up to time $T-1$ (that is, on ℓ_{T-1}), for every horizon $h \geq 1$,*

$$y_{T-1+h} \sim \mathcal{N}\big(\ell_{T-1}, \; \sigma^2 \, [1 + (h-1)\alpha^2]\big).$$

Proof. Iterating the state equation from $T-1$, $\ell_{T-2+h} = \ell_{T-1} + \alpha \sum_{j=1}^{h-1} \varepsilon_{T-1+j}$. Then

$$y_{T-1+h} = \ell_{T-2+h} + \varepsilon_{T-1+h} = \ell_{T-1} + \alpha \sum_{j=1}^{h-1} \varepsilon_{T-1+j} + \varepsilon_{T-1+h}.$$

The future innovations are independent of ℓ_{T-1} and of each other, with mean 0 and variance σ^2. The right-hand side is therefore Gaussian with mean ℓ_{T-1} and variance $\alpha^2(h-1)\sigma^2 + \sigma^2$. □

The point forecast is flat, but the uncertainty grows linearly in h. Each future shock moves the level permanently by $\alpha\varepsilon$, so the model is a random walk observed with noise. The parameter α sets how much of each shock is permanent ($\alpha\varepsilon_t$ enters the level) and how much is transitory.

2.1 Equivalence with ARIMA(0,1,1)

Proposition 2.2. *Under* (4)*, the differences* $\Delta y_t = y_t - y_{t-1}$ *follow the MA(1) process* $\Delta y_t = \varepsilon_t - (1-\alpha)\,\varepsilon_{t-1}$*, so* y *is ARIMA(0,1,1) with moving-average coefficient* $\theta = -(1-\alpha)$*.*

Proof. From (4), $y_t - y_{t-1} = (\ell_{t-1} + \varepsilon_t) - (\ell_{t-2} + \varepsilon_{t-1}) = (\ell_{t-1} - \ell_{t-2}) + \varepsilon_t - \varepsilon_{t-1} = \alpha\varepsilon_{t-1} + \varepsilon_t - \varepsilon_{t-1}$. □

The MA polynomial $1-(1-\alpha)z$ has its root at $1/(1-\alpha)$, outside the unit circle for $\alpha \in (0,2)$. The process is therefore invertible, and ε_t can be recovered from past y. This is what makes the recursion (1) forget its initialization (Proposition 1.1). `nabia` restricts α to $(0,1]$, where all weights in (3) are non-negative and the level is a genuine average.

2.2 Equivalence with the Kalman filter of the local level model

The local level (random walk plus noise) model has two independent noise sources:

$$x_t = x_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0,q), \qquad y_t = x_t + v_t, \quad v_t \sim \mathcal{N}(0,r). \tag{5}$$

Proposition 2.3 (Muth [2])**.** *The Kalman filter for* (5) *converges to the recursion* $\hat{x}_t = \hat{x}_{t-1} + K(y_t - \hat{x}_{t-1})$ *with steady-state gain*

$$K = \frac{P}{P+r}, \qquad P = \frac{q + \sqrt{q^2 + 4qr}}{2}.$$

This is SES with $\alpha = K$*. Conversely, the SES with factor* $\alpha \in (0,1)$ *is the steady state of* (5) *with signal-to-noise ratio* $q/r = \alpha^2/(1-\alpha)$*.*

Proof. For this model the predicted variance $P_t = \mathrm{Var}(x_t \mid y_{0:t-1})$ obeys the Riccati recursion $P_{t+1} = P_t r/(P_t + r) + q$ (see the Kalman filter note, Section 6). A fixed point satisfies $P(P+r) = Pr + q(P+r)$, that is $P^2 - qP - qr = 0$, whose positive root is the P above. The map $P \mapsto Pr/(P+r) + q$ is increasing and concave with slope $r^2/(P+r)^2 < 1$ at the fixed point, so the iteration converges to it from any $P_0 \geq 0$. The filtered estimate is $\hat{x}_t = \hat{x}_{t-1} + K_t(y_t - \hat{x}_{t-1})$ with $K_t = P_t/(P_t+r) \to K$. For the converse, $K = P/(P+r)$ gives $P = Kr/(1-K)$. Substituting into $P^2 = q(P+r)$ gives $K^2 r^2/(1-K)^2 = q\,r/(1-K)$, that is $q/r = K^2/(1-K)$. □

The `nabia` test suite checks Proposition 2.3 numerically. After 300 steps, the local-level Kalman filter and an SES with $\alpha = K$ agree to 10^{-9}.

3 Estimation

Innovation variance. Conditionally on the initial level, the likelihood of (4) factorizes over the one-step errors (prediction error decomposition): $\log L(\sigma^2) = -\frac{T}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_t e_t^2$. Setting the derivative in σ^2 to zero gives the maximum-likelihood estimate

$$\hat{\sigma}^2 = \frac{\mathrm{SSE}}{T}, \qquad \mathrm{SSE} = \sum_{t=0}^{T-1} e_t^2.$$

Smoothing factor. For fixed σ^2, maximizing the likelihood in α amounts to minimizing $\mathrm{SSE}(\alpha)$. This one-dimensional function is smooth but not necessarily convex. `nabia` leaves the choice to the user: the `forecast_demo` example scans a grid, and `NelderMead` on $[0, 1]$ or a golden-section search would also do.

4 Implementation

4.1 Interface

The class implements the forecaster interface

`IForecaster<SimpleExponentialSmoothingParams>` :

```
SimpleExponentialSmoothing model(0.3); // or (std::make_unique<Params>(...))
model.fit(y); // Eigen::VectorXd, finite values
Forecast fc = model.forecast(12); // mean and variance per step
Eigen::VectorXd lo = fc.lower(0.95), hi = fc.upper(0.95);
Eigen::VectorXd one_step = model.fitted_values(); // l_{t-1}, the forecast of y_t
double sse = model.sse();
```

4.2 Algorithm

Algorithm 1 SES fit and forecast

Require: $y_0, \ldots, y_{T-1}$ finite, $\alpha \in (0, 1]$

$\ell \leftarrow y_0$; $\mathrm{SSE} \leftarrow 0$ ▷ $\ell_{-1} = y_0$

for $t = 0, \ldots, T-1$ **do**

  $\hat{y}_t \leftarrow \ell$ ▷ one-step forecast, stored in `fitted_values()`

  $e \leftarrow y_t - \ell$; $\mathrm{SSE} \leftarrow \mathrm{SSE} + e^2$; $\ell \leftarrow \ell + \alpha e$

end for

$\hat{\sigma}^2 \leftarrow \mathrm{SSE}/T$

for $h = 1, \ldots, H$ **do** ▷ `forecast(H)`

  $\mu_h \leftarrow \ell$; $v_h \leftarrow \hat{\sigma}^2\,(1 + (h-1)\alpha^2)$

end for

Initialization. The implementation uses $\ell_{-1} = y_0$, so the first error is $e_0 = 0$ and the first fitted value equals the first observation. By Proposition 1.1 the effect of this choice decays like $(1-\alpha)^t$. Because $e_0 = 0$ by construction, $\hat{\sigma}^2 = \mathrm{SSE}/T$ averages $T-1$ informative errors over T terms. This slightly underestimates σ^2 in short samples, a bias of factor $(T-1)/T$.

Prediction intervals. `Forecast::lower(level)` and `upper(level)` return $\mu_h \mp z\sqrt{v_h}$ with $z = \Phi^{-1}((1 + \text{level})/2)$, computed by `nabia::stats::normal_quantile`. These are exact intervals under (4) with known parameters. They ignore the estimation error in α and σ^2, which makes them slightly too narrow in small samples.

Complexity and validation. Fitting is a single $O(T)$ pass with $O(T)$ memory (the fitted values), and forecasting is $O(H)$. The constructor rejects $\alpha \notin (0, 1]$, and `fit` rejects an empty or non-finite series. `forecast` before `fit` throws `std::logic_error`.

Correspondence with the code.

Mathematics	`SimpleExponentialSmoothing`
α	`parameters().alpha`
ℓ_{T-1}	private `level_`
$(\ell_{t-1})_t$	`fitted_values()`
SSE	`sse()`
(μ_h, v_h)	`forecast(H).mean`, `.variance`

Tests. `tests/test_exponential_smoothing.cpp` covers several cases: a hand-computed series (levels $10 \to 15 \to 22.5$, SSE $= 325$, variances $325/3$ and $1.25 \cdot 325/3$), $\alpha = 1$ as the naive forecast, invalid α, and forecasting before fitting. The Kalman filter equivalence of Proposition 2.3 is tested in `tests/test_forecasters.cpp`.

References

[1] R. G. Brown, *Statistical Forecasting for Inventory Control*, McGraw-Hill, 1959.

[2] J. F. Muth, Optimal properties of exponentially weighted forecasts, *J. Amer. Statist. Assoc.* 55 (1960) 299–306.

[3] E. S. Gardner, Exponential smoothing: the state of the art, *J. Forecasting* 4 (1985) 1–28.

[4] R. J. Hyndman, A. B. Koehler, J. K. Ord and R. D. Snyder, *Forecasting with Exponential Smoothing: The State Space Approach*, Springer, 2008.

[5] A. C. Harvey, *Forecasting, Structural Time Series Models and the Kalman Filter*, Cambridge University Press, 1989.